

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación

Programación Concurrente y Paralela

Práctica 2

Productor – Consumidor

(Primer parcial)

Equipo:

Islas Gómez Luis Antonio — 202372860
Ramírez Candia Alfredo — 202375947
Pérez Flores Julio César — 202247445
Navarro Soto Mario Alberto — 202264602

Docente: Hilda Castillo Zacatelco

11 de septiembre de 2026

Índice general

1	Introducción y objetivo	2
2	Marco teórico	2
3	Desarrollo	2
3.1	Estructura del programa	2
3.2	Candado atómico	2
3.3	Región crítica	3
4	Problemas de sincronización y condiciones de carrera	3
5	Solución planteada	3
6	Pruebas y resultados	4
7	Conclusiones	5

1 Introducción y objetivo

El problema productor–consumidor consiste en dos procesos que comparten un buffer: el productor deposita datos y el consumidor los retira. El objetivo de esta práctica es implementarlo únicamente con `fork()` y archivos de texto, sin emplear tuberías, señales, `wait`, `exec`, semáforos, mutex, monitores, memoria compartida ni colas de mensajes. La sincronización se resuelve con espera ocupada (ciclos vacíos) y retardos.

2 Marco teórico

- **Región crítica:** fragmento de código que accede a un recurso compartido y no debe ejecutarse simultáneamente por dos procesos.
- **Condición de carrera:** fallo producido cuando el resultado depende del orden no determinista en que los procesos acceden al recurso.
- **Exclusión mutua:** garantía de que solo un proceso está en la región crítica a la vez.
- **Espera ocupada:** el proceso que no puede avanzar repite un ciclo hasta que la condición cambia; se añaden pequeños retardos (`usleep`) para ceder tiempo de CPU.
- **COBEGIN / COEND:** notación que encierra bloques de código que pueden ejecutarse en paralelo.

3 Desarrollo

3.1 Estructura del programa

Se crea un proceso productor con `fork()`. El proceso padre actúa como consumidor y el hijo como productor. El buffer compartido es el archivo `buffer.txt` (una línea por elemento) y el candado es `candado.txt`. El productor agrega al final del archivo y el consumidor retira el primer elemento, por lo que se respeta el orden FIFO.

3.2 Candado atómico

La exclusión mutua se logra creando el archivo candado con la bandera `O_CREAT | O_EXCL`. El sistema operativo realiza esa creación de forma indivisible: si el archivo ya existe, la llamada falla con `EEXIST` y el proceso reintenta en un ciclo (espera ocupada). Al terminar, el candado se libera con `unlink()`.

```
1 void adquirirCandado(){
2     while(open(ARCHIVO_CANDADO, O_CREAT | O_EXCL | O_WRONLY, 0600) ==
3         -1){
4         usleep(1000);
5     }
```

```

6
7 void liberarCandado(){
8     unlink(ARCHIVO_CANDADO);
9 }

```

3.3 Región crítica

El conteo del buffer y su modificación se realizan *dentro* del candado, de modo que la verificación y la operación forman una sola acción atómica frente al otro proceso.

```

1  /* Productor */
2  adquirirCandado();
3  if(contarItems() < BUFFER_SIZE){
4      agregarItem(numero);
5      liberarCandado();
6      /* ... */
7  } else {
8      liberarCandado();
9  }

```

4 Problemas de sincronización y condiciones de carrera

Al compartir recursos mediante archivos surgen los siguientes riesgos, que la solución presentada debe evitar:

1. **Condición de carrera en el acceso concurrente.** Si los dos procesos leen y modifican el archivo sin coordinarse, sus operaciones se intercalan y el resultado depende del orden de ejecución.
2. **Falta de atomicidad del candado.** Un candado construido leyendo y luego escribiendo el archivo deja una ventana entre ambas operaciones; ambos procesos podrían encontrar el candado libre y entrar a la vez. Por eso se requiere una primitiva indivisible.
3. **Verificación y modificación separadas.** Comprobar el conteo del buffer antes de acceder a él produce una condición *check-then-act*: entre la comprobación y la operación el buffer puede cambiar por el otro proceso.
4. **Pérdida de actualizaciones.** Como `quitarItem()` reescribe todo el archivo, un agregado que ocurra entre la lectura y la reescritura se perdería. Igualmente, el productor podría desbordar el buffer o el consumidor extraer de un buffer vacío.

5 Solución planteada

La solución se basa en dos ideas: el candado se toma mediante una operación atómica del sistema de archivos y toda verificación y modificación del buffer se realiza dentro de la región crítica. El

siguiente fragmento muestra el algoritmo implementado, encerrado entre COBEGIN y COEND para representar los dos procesos que corren en paralelo.

```
1 // Variables compartidas: buffer.txt (buffer) y candado.txt (candado).
2
3 // COBEGIN
4 //Productor (proceso hijo) */
5 int numero = 1;
6 while(1){
7     adquirirCandado();
8     if(contarItems() < BUFFER_SIZE){
9         agregarItem(numero);
10        liberarCandado();
11        printf("[Productor] genero: %d\n", numero);
12        numero = (numero % 10) + 1;
13    } else {
14        liberarCandado();
15    }
16    usleep(200000);
17 }
18
19 // Consumidor (proceso padre)
20 while(1){
21     adquirirCandado();
22     int item = -1;
23     if(contarItems() > 0){
24         item = quitarItem();
25     }
26     liberarCandado();
27     if(item != -1){
28         printf("\t\t[Consumidor] tomo: %d\n", item);
29     }
30     usleep(500000);
31 }
32 // COEND
```

Con esto, adquirir/liberar garantizan exclusión mutua y las verificaciones de lleno/vacío ya no compiten con el otro proceso.

6 Pruebas y resultados

Se compiló con `gcc ComePica.c -o bin/comepica -Wall -Wextra` sin errores y se ejecutó durante algunos segundos. La salida muestra que el productor nunca supera los 10 elementos y el consumidor solo retira cuando hay datos; el orden de retiro es FIFO.

```
1 [Productor] genero: 1
2 [Productor] genero: 2
3 [Productor] genero: 3
```

```
4         [Consumidor] tomo: 1
5 [Productor] genero: 4
6 ...
7         [Consumidor] tomo: 5
```

7 Conclusiones

El uso de archivos de texto como recurso compartido es suficiente si se construye un candado atómico y se define con precisión la región crítica. La principal enseñanza es que no basta con “bloquear” el acceso: separar la verificación de la operación deja abierta una condición de carrera. Colocar ambas acciones bajo el candado y usar una primitiva indivisible del sistema de archivos elimina las pérdidas de actualizaciones y respeta los límites del buffer.